Proyecto final II: Informe de síntesis y reflexión

En el Proyecto final I desarrollé y probé tres servicios back-end en Java, sin interfaz gráfica ni base de datos, estos son los servicios de contactos, tareas y citas. Se transformó cada requisito en casos de prueba JUnit, separando la validación del objeto de las operaciones del servicio. Utilicé datos válidos, entradas inválidas y situaciones límite para comprobar no solo que el programa aceptara operaciones correctas, sino también que rechazara estados inconsistentes.

Para contactos, ContactTest.java comprobó las restricciones de identificador, nombres, teléfono y dirección. Por ejemplo, testPhoneMustHaveExactlyTenDigits se orientó al requisito de diez dígitos, mientras que testContactIdCannotBeNull verificó la obligatoriedad del identificador. ContactServiceTest.java probó agregar, eliminar y actualizar contactos, incluida la prohibición de identificadores duplicados.

En tareas, TaskTest.java incluyó testMaximumLengths, testNullFields y testInvalidUpdates para comprobar límites de 10 caracteres en el ID, 20 en el nombre y 50 en la descripción, así como actualizaciones inválidas. TaskServiceTest.java abordó altas, duplicados, bajas, modificaciones y elementos inexistentes; testInvalidUpdates ayudó a comprobar que un dato rechazado no alterara indebidamente el objeto. En citas, AppointmentTest.java incluyó testInvalidAppointmentDate para detectar fechas nulas o anteriores al presente y testInvalidDescription para revisar la longitud de la descripción. AppointmentServiceTest.java verificó altas, duplicados y eliminaciones, sin incorporar actualizaciones que el cliente no exigía.

Para evaluar la eficacia de mis pruebas, medí la cobertura en Eclipse mediante EclEmma/JaCoCo. La cobertura general reportada fue del 90,3 % en contactos, del 80,8 % en tareas y del 86,0 % en citas. Al examinar específicamente el código de producción, Contact.java

y ContactService.java obtuvieron un 100 % cada uno; Task.java y TaskService.java, también un 100 % cada uno; Appointment.java, un 100 %, y AppointmentService.java, un 90 %. Por tanto, las seis clases de producción superaron el 80 % solicitado en el Proyecto final I. La cobertura de los propios archivos de pruebas fue diferente, ContactTest.java, 89,6 %; ContactServiceTest.java, 82,4 %; TaskTest.java, 67,6 %; TaskServiceTest.java, 80,5 %; AppointmentTest.java, 69,7 %, y AppointmentServiceTest.java, 93,1 %. Estas cifras no deben confundirse con la cobertura de las clases evaluadas. Aunque los resultados respaldan la amplitud de las pruebas, la cobertura por sí sola no garantiza que todas las comprobaciones sean correctas ni que se hayan explorado todos los escenarios posibles (JaCoCo, s. f.).

Al escribir las pruebas busqué solidez técnica mediante aserciones explícitas como assertEquals(...) para comparar resultados y assertThrows(...) para exigir excepciones ante entradas inválidas, de acuerdo con las capacidades de JUnit (Bechtold et al., s. f.). Como líneas concretas de mis pruebas, en TaskTest.java empleé new Task("T001", "Estudiar", "Estudiar Java") para crear un caso válido; en ContactTest.java, new Contact("1234567890", "Bastian", "Garcia", "1234567890", "Temuco Chile") sirvió para verificar los datos iniciales. Métodos como testNullFields contrastaron los casos negativos. En TaskServiceTest.java, la preparación @BeforeEach public void setUp() { service = new TaskService(); } creaba un servicio limpio antes de cada prueba. Esta decisión mejoró la eficiencia, ya que se evitó repetir la inicialización, reduje dependencias entre pruebas y mantuve las comprobaciones enfocadas.

Las técnicas que más utilicé fueron pruebas unitarias dinámicas de caja negra, partición de equivalencia y análisis de valores límite. La partición separó entradas admisibles de entradas nulas o inválidas, los límites permitieron examinar longitudes máximas y valores inmediatamente superiores. También contrasté flujos positivos y negativos, por ejemplo, agregar

una tarea nueva frente a intentar duplicar su ID, o crear una cita futura frente a una vencida. Estas técnicas parten de los requisitos observables y favorecen casos representativos sin probar todas las entradas posibles (International Software Testing Qualifications Board [ISTQB], s. f.).

No apliqué pruebas de integración con bases de datos, pruebas de interfaz, carga ni seguridad, porque el alcance consistía en clases aisladas y almacenamiento en memoria. En otro proyecto, las pruebas de integración permitirían detectar fallas entre servicios y persistencia, las de interfaz revelarían problemas en la interacción del usuario, las de rendimiento evaluarían tiempos de respuesta con numerosos registros, y las de seguridad examinarían entradas maliciosas o accesos indebidos. En una aplicación real de citas, por ejemplo, la integración debería comprobar que una reserva persistiera correctamente y fuera visible para otros usuarios. Las pruebas de carga serían importantes si cientos de personas reservaran simultáneamente, mientras que las de seguridad protegerían los datos personales. Tampoco realicé una revisión estática formal por pares, que sería útil para detectar problemas de legibilidad y diseño que las pruebas ejecutables no muestran. Combinar técnicas permite abordar riesgos que una prueba unitaria, por sí sola, no puede resolver.

Adopté una actitud de precaución al intentar refutar el comportamiento esperado, en vez de limitarme a confirmar que los casos normales funcionaban. Fue importante comprender las interrelaciones: un ID duplicado afecta la colección completa, una actualización inválida no debe corromper el valor anterior; y una cita exige relacionar la fecha ingresada con el instante actual. Por eso probé tanto reglas de los objetos como operaciones de los servicios. El caso de la validación telefónica mostró que una prueba sirve cuando también descubre una modificación defectuosa.

Al probar mi propio código reconocí el riesgo del sesgo de confirmación, ya que, como desarrollador, puedo dar por válidas decisiones que yo mismo implementé. Para reducirlo, derivé los casos directamente de los requisitos y añadí situaciones contrarias a mis suposiciones, como campos nulos, límites excedidos, duplicados y eliminaciones inexistentes. En un equipo profesional reforzaría este control con revisión cruzada y criterios de aceptación independientes del autor del código.

Omitir una prueba de excepción puede parecer un ahorro, pero permite introducir defectos que luego exigen correcciones más costosas. Un cambio aparentemente pequeño en la longitud permitida para un campo podría invalidar varias operaciones del servicio; las pruebas automatizadas permiten advertirlo antes de entregar. Siento que es necesario mantener pruebas repetibles, actualizarlas cuando cambien los requisitos, volver a medir su cobertura tras cada cambio y revisar las rutas críticas antes de entregar. También documentaría cualquier limitación conocida. Así, las pruebas pasan de ser un trámite académico a una protección continua de la confiabilidad del software.

Referencias

Bechtold, S., Brannen, S., Link, J., Merdes, M., Philipp, M., Rancourt, J., & Stein, C. (s.f.). *JUnit 5 user guide (versión 5.12.0)*. Obtenido de JUunit: https://docs.junit.org/5.12.0/user-guide/

International Software Testing Qualification Board. (s.f.). *ISTQB glossary*. Obtenido de ISTQB: https://glossary.istqb.org/

JaCoCo. (s.f.). *Coverage counters*. Obtenido de JaCoCo: https://www.jacoco.org/jacoco/trunk/doc/counters.html